

# Predicting Corporate Bond YTM Changes Using Machine Learning

Team members: Yuxi Geng, Yutung Lin, Qinqin Huang

## 1. Objective

The objective of this project is to predict corporate bond yield to maturity (YTM) changes using firm-level fundamentals, market-based risk factors, and macroeconomic indicators.

By applying various machine learning regression models such as ElasticNet, Boosting and Multi-layer Perceptron, we aim to improve prediction accuracy compared to traditional linear regression models.

In addition, we used the change direction of YTM movements as a classification target since there are many outliers in the YTM change values that may affect regression performance. We applied classification models such as Logistic Regression, ElasticNet Classifier, and Boosting Classifier to predict the direction of YTM changes.

## 2. Methodology

### 2.1 Data Description

We will utilize data from Wharton Research Data Services (WRDS) to construct our dataset. The data frequency is monthly and data range is from July 2002 to February 2025.

**2.1.1 Target Variable** The target variable is the corporate bond yield to maturity (YTM) change, defined as the difference in YTM over a one-month horizon:

$$\Delta YTM_i = YTM_{i,t} - YTM_{i,t-1}$$

where  $YTM_{i,t}$  is the yield to maturity of bond  $i$  at the end of month  $t$ .

For classification, the target variable is defined as:

$$YTM\_Direction_i = \begin{cases} up, & \text{if } \Delta YTM_i > 1e-6 \\ neutral, & \text{if } \Delta YTM_i \approx 0 \\ down, & \text{if } \Delta YTM_i < -1e-6 \end{cases}$$

The Data is from the “WRDS Bond Returns” database, which provides monthly bond yields and other bond characteristics, sourced from TRACE and Mergent FISD. The dataset has been cleaned and more suitable for analysis.

We only consider senior bonds and exclude defaulted bonds. Additionally, the yields exceeding  $[-1, 1]$  are treated as outliers and removed from the dataset.

Total number of observations meeting the criteria is approximately 1760852, covering 74191 unique bonds and 3187 unique firms over the sample period.

After merging with firm-level and macroeconomic data, the final dataset contains 12117 unique bonds.

The data descriptive statistics of the yield to maturity (YTM) changes are as follows:

```
count    854769.000000
mean      -0.000005
std        0.014440
min       -0.980800
25%       -0.001840
50%       -0.000030
75%        0.001710
max        0.979029
Name: ytm_chg, dtype: float64
```

The number of observations for each YTM change direction category is as follows:

```
down      435398
up         399205
neutral    20166
Name: count, dtype: int64
```

### 2.1.2 Features

#### 1. Key Columns

- Bond Identifier: CUSIP
- Firm Identifier: PERMNO
- Date: End of Month

We have checked the missingness and uniqueness of these key columns.

#### 2. Bond-level features

- Time to Maturity (tmt)
- Coupon Rate (coupon)
- t\_spread: the weighted average bid-ask spread of the bond which measures liquidity.
- return\_eom: bond return of last month.
- Credit Ratings: One-hot encoded variables for ratings from AAA to D (rating\_A, rating\_AA, ..., rating\_D). The ratings are weighted averages ratings from WRDS Bond Returns database rather than S&P, Moody's, or Fitch alone.

- Rating Change Indicators: Binary variables indicating whether there was an upgrade or downgrade in the bond’s credit rating in the past month (upgrade, downgrade).

Here, we used lagged values of features except time to maturity and coupon rate to avoid data leakage.

| Variable   | Count   | Mean | Std  | Min   | 25%  | 50%  | 75%  | Max    |
|------------|---------|------|------|-------|------|------|------|--------|
| tmt        | 3.56e+6 | 6.35 | 8.67 | 0.00  | 1.61 | 3.54 | 6.89 | 102.13 |
| coupon     | 3.56e+6 | 0.03 | 0.03 | 0.00  | 0.00 | 0.03 | 0.06 | 0.34   |
| t_spread   | 1.79e+6 | 0.01 | 0.01 | 0.00  | 0.00 | 0.00 | 0.01 | 2.00   |
| yield      | 3.39e+6 | 0.01 | 0.13 | -1.00 | 0.00 | 0.03 | 0.05 | 1.00   |
| ret_eom    | 3.42e+6 | 0.01 | 1.18 | -1.00 | 0.00 | 0.00 | 0.01 | 2090.0 |
| rating_A   | 3.42e+6 | 0.20 | 0.40 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_AA  | 3.42e+6 | 0.04 | 0.21 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_AAA | 3.42e+6 | 0.01 | 0.09 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_B   | 3.42e+6 | 0.03 | 0.16 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_BB  | 3.42e+6 | 0.04 | 0.20 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_BBB | 3.42e+6 | 0.23 | 0.42 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_C   | 3.42e+6 | 0.00 | 0.02 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_CC  | 3.42e+6 | 0.00 | 0.03 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_CCC | 3.42e+6 | 0.01 | 0.09 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| rating_D   | 3.42e+6 | 0.00 | 0.02 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| upgrade    | 3.42e+6 | 0.00 | 0.04 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |
| downgrade  | 3.42e+6 | 0.00 | 0.05 | 0.00  | 0.00 | 0.00 | 0.00 | 1.00   |

### 3. Firm-level features (yutung)

We incorporate firm-level information at both the daily and quarterly frequency.

- Daily Stock Price and Returns: Includes the daily closing stock price (PRC), daily stock return (RET), and trading volume (VOL). These variables capture short-term market performance and trading activity.
- Daily Trading Liquidity Measures: Includes the daily bid and ask quotes (BIDLO, ASKHI) and the number of trades (NUMTRD) as indicators of market liquidity.
- Quarterly Accounting Fundamentals: Includes total assets (atq), total liabilities (ltq), common equity (ceqq), shares outstanding (cshoq), and quarterly net income (niq). These variables summarize the firm’s financial position and profitability at the quarterly reporting date.
- Firm Identification: The PERMNO identifier is present in both daily stock data and quarterly fundamental data, enabling direct alignment of firm-level information across time.

### 4. Macroeconomic features (yuxi)

## 2.2 Feature Construction (Yutung)

After preparing the bond-level, firm-level, sector-level, and macroeconomic datasets, we align all information at a monthly frequency and construct the final panel used for modeling. This stage focuses on dataset merging, temporal alignment, lagging to avoid look-ahead bias, missing-value handling, and feature normalization.

1. Merging Procedure We combine the datasets in the following order to maintain consistent identifiers and time alignment: (1.) Stock + Fundamentals (PERMNO, date); CRSP monthly stock aggregates are merged with Compustat monthly-filled accounting fundamentals using the common firm identifier (PERMNO) and month-end date. (2.) Add Macroeconomic Variables (date): Monthly macro data are merged using the calendar month-end date. (3.) Add Sector ETF Information (industry code): Each firm is mapped to its sector ETF using global industry classification, and monthly ETF returns/prices are merged accordingly. (4.) Merge with Bond Data (CUSIP root, date): Bonds are linked to issuers via the first six digits of CUSIP (issuer6). Only issuer-month pairs that appear in both datasets are retained to ensure valid alignment.

2. Missing-Value Handling After merging, missing values are handled according to variable type:

Categorical Variables: • Credit rating dummies (AAA...D) • Rating transition indicators • Compustat data-status flag (costat) Here we use cross-sectionally monthly median to impute missing values, preserving the categorical distribution across firms each month.

Numeric Variables: We also use cross-sectionally monthly median to impute missing values for numeric variables, then for remaining NaNs, we set them to zero.

3. Normalization (1.) Rank-Normalized Features • Includes stock features, fundamentals, derived ratios, growth variables, and bond-level numeric predictors. • Each month, variables are transformed using:

$$\text{scaled\_rank} = 2 \times \frac{\text{rank}}{N} - 1$$

producing values in  $[-1, 1]$ . • For binary variables (e.g., upgrade/downgrade), ranks are assigned such that 1 maps to 1 and 0 maps to -1.

(2.) Not Normalized (kept in raw form) Includes macroeconomic indicators and yield-curve variables: • sp500\_ret, gdp\_gr, cpi\_infl • ir3m\_chg, ir10y\_chg • vix\_chg • gs3m, term\_spread These variables are identical across all firms in a given month; rank-normalizing them collapses them into constants, eliminating their informational content. Therefore, they are kept in raw form to preserve meaningful macro signals.

## 2.3 Data Splitting

In this project, we used a rolling window forecasting approach similar to our homework. At each year, we used a time-series split to create training, validation, and test sets. The training set consisted of 10 years of data, the validation set and test set each consisted of 1 year of data. When tuning is not involved, the validation set was combined with the training set to train the final model. For example, to predict YTM changes in 2012, we used data from 2002 to 2011 as the training set, data from 2012 as the validation set, and data from 2013 as the test set. Then, to predict YTM changes in 2013, we used data from 2003 to 2012 as the training set, data from 2013 as the validation set, and data from 2014 as the test set. This process was repeated until we predicted YTM changes in 2024.

## 2.4 Modeling Framework

### 2.4.1 Regression Models

We implemented and compared the following machine learning models for regression task:

| Model                  | Description                |
|------------------------|----------------------------|
| Linear Regression      | Baseline linear model      |
| ElasticNet             | Combination of L1 and L2   |
| Boosting (LightGBM)    | Ensemble of weak learners  |
| Multi-layer Perceptron | Feedforward neural network |

Here, for the ElasticNet model, we tuned the hyperparameters `alpha` and `l1_ratio` using 10-fold cross-validation on the training set. However, we only tuned once based on the first training set and used the same hyperparameters for all other training sets to reduce computation time. This approach was applied to all tuning processes in this project. For the Boosting model, we tuned the hyperparameters using a grid search.

We also implemented a Stacked Regressor that combines predictions from Linear Regression, ElasticNet, and LightGBM models using linear regression as the meta-model to improve overall performance.

For the Multilayer Perceptron (MLP), we first standardized all continuous predictors using the training-set mean and standard deviation, and clipped extreme standardized values to a bounded range before feeding them into the network. The MLP is a small fully connected feedforward neural network with ReLU activation, trained with mean squared error loss and the Adam optimizer using early stopping based on validation loss. Due to the higher computational cost of neural networks, the MLP was estimated on a separate 80/20 time split rather than within the rolling-window framework, so its results are reported as a complementary single-split experiment rather than being directly comparable to the rolling-window averages of the other regression models.

The MLP is not part of the stacked regressor due to its separate training procedure and high computational cost.

### 2.4.2 Classification Models

For classification task, we implemented and compared the following models:

| Model                          | Description                 |
|--------------------------------|-----------------------------|
| Logistic Regression            | Linear classification model |
| ElasticNet Classifier          | Combination of L1 and L2    |
| Boosting Classifier (LightGBM) | Ensemble of weak learners   |

Here, we also tuned hyperparameters for ElasticNet Classifier and Boosting Classifier using similar approaches as in regression task.

For the Stacked Classifier, we combined predictions from Logistic Regression, ElasticNet Classifier, and LightGBM Classifier using logistic regression as the meta-model.

## 2.5 Model Evaluation

We used a rolling window evaluation approach and calculated performance metrics on the test set for each year from 2012 to 2024. Then, the metrics were averaged to assess overall model performance.

For regression models, we evaluated performance using:

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Median Absolute Error (MedAE)
- $R^2$

For classification models, we evaluated performance using:

- Accuracy: Overall correctness of the model.
- Precision: Proportion of positive identifications that were actually correct.
- Recall: Proportion of actual positives that were identified correctly.

## 3. Results and Discussion

### 3.1 Regression Results

The table below summarizes the performance of different regression models on the test set:

| Model                  | MSE      | MAE      | MedAE    | $R^2$    |
|------------------------|----------|----------|----------|----------|
| Elastic Net Regression | 0.000093 | 0.003167 | 0.002032 | 0.005956 |
| LGBM Regressor         | 0.000084 | 0.002817 | 0.001607 | 0.085135 |

| Model                 | MSE      | MAE      | MedAE    | $R^2$    |
|-----------------------|----------|----------|----------|----------|
| Linear Regression     | 0.000093 | 0.003171 | 0.002036 | 0.004982 |
| Stacked Regressor     | 0.000084 | 0.002841 | 0.001607 | 0.063947 |
| Multilayer Perceptron | 0.000103 | 0.003265 | 0.001792 | 0.232484 |

The Elastic Net Regression model did not significantly outperform the Linear Regression model, indicating that regularization may not provide substantial benefits in this context. The LGBM Regressor achieved the lowest MSE and highest  $R^2$ , suggesting that ensemble methods can better capture complex relationships in the data. The Stacked Regressor also performed well, leveraging the strengths of multiple models but did not surpass the LGBM Regressor.

The multilayer perceptron (MLP) attains test errors between those of the linear models and LightGBM, providing additional evidence that some nonlinear structure is present in YTM changes, although tree-based ensembles appear to exploit it more effectively in our current setup.

### 3.2 Classification Results

As we mentioned before, we defined the target variable for classification as the direction of YTM changes (up, neutral, down) to mitigate the impact of outliers. The table below summarizes the performance of different classification models on the test set:

| Model                      | Accuracy | Precision | Recall   |
|----------------------------|----------|-----------|----------|
| Elastic Net Classification | 0.682760 | 0.694127  | 0.682760 |
| LGBM Classifier            | 0.713273 | 0.722421  | 0.713273 |
| Logistic Regression        | 0.682736 | 0.694145  | 0.682736 |
| Stacked Classifier         | 0.706753 | 0.718828  | 0.706753 |

In classification tasks, both the Elastic Net Classifier and Logistic Regression models exhibited similar performance, which is consistent with the regression results. The LGBM Classifier outperformed the other models, achieving the highest accuracy, precision, and recall. The Stacked Classifier also showed strong performance, although it did not exceed the LGBM Classifier.

## 4. Conclusion

In this project, we explored various machine learning models to predict corporate bond yield to maturity changes using firm-level fundamentals, market-based risk factors, and macroeconomic indicators. Our findings indicate that ensemble methods like LightGBM outperform traditional linear models in both regression and classification tasks. Regularization techniques such as ElasticNet did not yield significant improvements over linear models in this context. However, the

improvement was modest, suggesting that further enhancements such as more accurate data, additional features, or alternative modeling approaches may be necessary to achieve substantial gains in predictive accuracy. textual disclosures, market microstructure signals), or alternative modeling frameworks that more directly account for regime shifts and rare credit events.

## Appendix:

### A.1 Implementation Details

#### 1. Software and Libraries

- CPU: MacBook Pro (Apple M4 Pro)
- Programming Language: Python 3.13
- Libraries: pandas, numpy, scikit-learn, lightgbm, tensorflow/keras
- Code Repository: Github [[https://github.com/QinqinAndMacaulayCat/ml\\_project/tree/main](https://github.com/QinqinAndMacaulayCat/ml_project/tree/main)]

#### 2. Computation Time For Each Model (Approximate)

| Model                      | Tuning                      | Time for Training and Prediction |
|----------------------------|-----------------------------|----------------------------------|
| Linear Regression          |                             | 3.9 seconds                      |
| Elastic Net Regression     |                             | 1 minute 10 seconds              |
| LGBM Regressor             | 31 minutes<br>41.4 seconds  | 58.3 seconds                     |
| Stacked Regressor          |                             | 6 minutes 50 seconds             |
| Logistic Regression        |                             | 1 minute 30 seconds              |
| Elastic Net Classification | 83 minutes<br>59.8 seconds  | 23 minutes 40.7 seconds          |
| Boosting Classifier        | 114 minutes<br>48.8 seconds | 1 minute 27.9 seconds            |
| Stacked Classifier         |                             | 107 minutes 11.9 seconds         |

### A.2 Classification Confusion Matrix

The confusion matrixs are as follows:

#### 1. Logistic Regression

| actual \ predicted | up     | down   | neutral |
|--------------------|--------|--------|---------|
| <b>up</b>          | 155404 | 103686 | 264     |
| <b>down</b>        | 54596  | 205732 | 322     |
| <b>neutral</b>     | 3986   | 6213   | 1259    |

#### 2. Elastic Net Classification



| actual \ predicted | up     | down   | neutral |
|--------------------|--------|--------|---------|
| <b>up</b>          | 155442 | 103662 | 250     |
| <b>down</b>        | 54610  | 205719 | 321     |
| <b>neutral</b>     | 3990   | 6218   | 1250    |

### 3. LGBM Classifier

| actual \ predicted | up     | down   | neutral |
|--------------------|--------|--------|---------|
| <b>up</b>          | 174264 | 84514  | 576     |
| <b>down</b>        | 58838  | 201267 | 545     |
| <b>neutral</b>     | 2317   | 3105   | 6036    |

### 4. Stacked Classifier

| actual \ predicted | up     | down   | neutral |
|--------------------|--------|--------|---------|
| <b>up</b>          | 176566 | 82735  | 53      |
| <b>down</b>        | 62485  | 198119 | 46      |
| <b>neutral</b>     | 3147   | 4741   | 3570    |